

A Single Generalization of `std::invoke`, `std::apply`, and `std::visit`

ISO/IEC JTC1 SC22 WG21

Document Number: P0376R0

Audience: Library Evolution Working Group

Matt Calabrese (metaprogrammingtheworld@gmail.com)

2016-05-28

Abstract

This paper presents a single generalization of `std::invoke`, `std::apply`, and the proposed `std::visit` [\[1\]](#) by specifying a template named `std::call` that takes a standard `Callable` followed by any number of arguments and/or "argument providers," the latter of which act as descriptions of portions of an argument list that are to be substituted in-place when the `Callable` is invoked. Each generated argument list portion may be any number of arguments with any combination of types, and the precise argument list and its values may even depend on runtime data. Due to the generality of the substitution mechanism, the facility allows argument providers such as ones that can unpack tuples at arbitrary positions in a larger argument list, as well as argument providers that can forward along the active field of a variant. In addition to these familiar operations, further argument providers are presented that do not directly correspond to existing library facilities.

Overview

The ideas described in this paper are relatively simple to understand at a high level and aid both in the expressive power of C++ and in code readability. Before going into more detailed motivating cases and implementation, the facility is easiest to describe with a set of usage examples. In the following `main` function, each invocation of `output_values` is given an equivalent argument list, although each time it is formed in a different way.

```
// This is just a Callable we will use for all of the following examples.
// It takes a stream and a series of arguments, outputting each.
constexpr auto output_values =
    [](auto& os, const auto&... args) -> decltype(auto) {
        return (os << ... << args);
    };

// In the following function, every time std::call is used, output_values is
// ultimately invoked with equivalent values.
int main() {
    // In its simplest usage, std::call does what std::invoke does.
    // It just invokes the Callable with the specified arguments.
    std::call(output_values, std::cout, 5, 3.5, std::string("Hello"));

    // It can also be used with a provider that unpacks tuples.
    // Note that std::cout is not a member of the tuple, so it is
    // simply forwarded to the Callable as-is.
    {
        auto args = std::make_tuple(5, 3.5, std::string("Hello"));
        std::call(output_values, std::cout, std::unpack(args));
    }

    // It can also unpack a tuple, followed by additional arguments.
    // Note that neither std::cout nor the string are members of the tuple.
    {
        auto args = std::make_tuple(5, 3.5);
        std::call(output_values, std::cout, std::unpack(args), std::string("Hello"));
    }

    // It can unpack multiple tuples in succession.
    {
        std::tuple<std::ostream&, int> head_args(std::cout, 5);
        auto tail_args = std::make_tuple(3.5, std::string("Hello"));
        std::call(output_values, std::unpack(head_args), std::unpack(tail_args));
    }

    // It can pass along a reference to the currently active field of a variant.
    {
        std::variant<int, std::string> arg0 = 5, arg2 = std::string("Hello");
        std::call(output_values, std::cout, std::active_field_of(arg0),
                  3.5,
                  std::active_field_of(arg2));
    }

    // It can access a tuple with a runtime index.
    {
```

```

auto args = std::make_tuple(5, std::string("Hello"));
std::call(output_values, std::cout, std::access_tuple(args, 0),
          3.5,
          std::access_tuple(args, 1));
}

// It can work with multiple different kinds of providers in the same call.
{
    std::variant<int, std::string> head_arg = 5;
    auto tail_args = std::make_tuple(3.5, std::string("Hello"));
    std::call(output_values, std::cout, std::active_field_of(head_arg), std::unpack(tail_args));
}

// It can work with providers that are composed
// (i.e. forward along the active field of each element
// of an unpacked tuple of variants).
{
    using variant_t = std::variant<double, std::string>;
    auto tail_args = std::make_tuple(variant_t(3.5), variant_t(std::string("Hello")));
    std::call(output_values, std::cout, 5, std::unpack(tail_args) | std::active_field_of);
}

// It can deduce a return type for arbitrarily complex compositions, including
// ones that involve variant access or tuple access with a run-time index.
// By default, if all possible paths do not have the same exact return type,
// then substitution will fail.
{
    std::variant<int, std::string> arg0 = 5, arg2 = std::string("Hello");
    auto& result = std::call(output_values, std::cout, std::active_field_of(arg0),
                           3.5,
                           std::active_field_of(arg2));
    static_assert(std::is_same_v<decltype(result), std::ostream&>);
}

// A return type can be explicitly specified to avoid automatic deduction.
// This is useful to force a different type or simply to reduce compile-times
// when dealing with sufficiently complicated invocations (such as access of
// several different variants in a single call).
{
    std::variant<int, std::string> arg0 = 5;
    std::call<void>(output_values, std::cout, std::active_field_of(arg0), 3.5, std::string("Hello"));
}

// A return type deducer can be explicitly specified if the default behavior
// is not suitable. A deducer is just a variadic template that is internally
// instantiated with the return type of each potential invocation.
{
    std::variant<int, std::string> arg0 = 5;
    std::call<some_user_defined_common_type_t>(
        output_values, std::cout, std::active_field_of(arg0), 3.5, std::string("Hello"));
}
}

```

Motivation

The primary motivation for a facility such as this comes from the desire for developers to reuse existing functions whenever possible without having to manually create lambdas. This comes up frequently in order to do things such as unpack only a portion of an argument list (as is often necessary when using `std::apply`), or forward along the active field of a variant to only one argument of a larger parameter list (as is often necessary when using facilities like the proposed `std::visit`). Expanding tuples in-place in an argument list is not an uncommon practice in other mainstream languages, such as Python [\[2\]](#), and forwarding the active field of a variant to an existing function is also not unheard of, even in statically-typed languages (an example of this is the dispatch operator of the Clay programming language [\[3\]](#)).

Complications of `std::apply`

Assuming the `output_values` function that was presented earlier, consider what is required by users to invoke the function when all but the stream argument are an element of a tuple. The following is an example using `std::apply` followed by what is required when using `std::call`. The version using the proposed `std::call` is more concise and considerably more readable:

```

auto args = std::make_tuple(5, 3.5, std::string("Hello"));

// This is what is required using std::apply.
std::apply([](const auto&... args) -> decltype(auto) {
    return output_values(std::cout, args...);
}

```

```
},
args);
```

```
// This is what is required using the proposed std::call.
std::call(output_values, std::cout, std::unpack(args));
```

Complications of std::visit

A similar kind of situation comes up when dealing with variants. Consider the following more tangible example, which is based on real-world code.

```
struct line { /*...*/ };
struct circle { /*...*/ };
struct square { /*...*/ };

struct in_collision_fun {
    // Each of these returns true if the arguments are in collision
    bool operator()(line, line) const { /*...*/ }
    bool operator()(line, circle) const { /*...*/ }
    // ... similar for each combination ...
} constexpr in_collision{};

int main() {
    circle my_circle(/*...*/);
    square my_square(/*...*/);

    variant<line, circle, square> my_circle_variant = my_circle,
                                my_square_variant = my_square;

    // This block uses std::visit
    {
        // Both arguments are variants.
        std::visit(in_collision, my_square_variant, my_circle_variant);

        // The first argument is an expanded variant.
        std::visit([&my_circle](const auto& first) {
            return in_collision(first, my_circle);
        }, my_square_variant);

        // The second argument is an expanded variant.
        std::visit([&my_square](const auto& second) {
            return in_collision(my_square, second);
        }, my_circle_variant);
    }

    // This block uses the proposed std::call
    {
        // Both arguments are variants.
        std::call(in_collision, std::active_field_of(my_square_variant),
                  std::active_field_of(my_circle_variant));

        // The first argument is an expanded variant.
        std::call(in_collision, std::active_field_of(my_square_variant),
                  my_circle);

        // The second argument is an expanded variant.
        std::call(in_collision, my_square,
                  std::active_field_of(my_circle_variant));
    }
}
```

Variant Deserialization

Serialization and deserialization of a variant is frequently done by serializing the integer discriminator of the variant, followed by serializing the corresponding field. Deserialization works by deserializing the integer discriminator and then deserializing an instance of the field type that corresponds to the discriminator. While this is very simple to think about at a high level, this deserialization process is not directly implementable using a facility like `std::visit`. It is actually surprisingly complicated to implement in a generic manner without additional facilities akin to the accepted-but-never-added Boost.Switch Library [\[4\]](#). However, this deserialization process can be easily implemented with `std::call`. In order to implement this functionality, the developer can use an argument provider that generates a `std::integral_constant` based on a runtime value (an argument provider named `std::to_constant_discriminator` is specifically included for this kind of purpose). An example of this can be seen below:

```
template <class Archive, class V>
void serialize_variant(Archive& archive, const V& v) {
```

```

serialize(archive, v.which());
std::call(serialize, archive, std::active_field_of(v));
}

// A function that deserializes into a variant when the field
// discriminator is known at compile-time.
// "discriminator" here is an instantiation of std::integral_constant.
constexpr auto deserialize_variant_field =
    [](auto& archive, auto& variant_, auto discriminator) {
        variant_.template emplace<discriminator.value>();
        deserialize_into(archive, std::get<discriminator.value>(variant_));
    };

template <class Archive, class V>
void deserialize_variant(Archive& archive, V& variant_) {
    std::call(deserialize_variant_field,
        archive,
        variant_,
        std::to_constant_discriminator<V>(deserialize<std::size_t>(archive)));
}

```

Proposed Facilities

Due to the nature of the templates involved, it is rather difficult to express a complete interface specification. What follows is an informal specification that should evolve considerably if the functionality that `std::call` provides is deemed valuable by the committee.

```

// If all of T... are the same type as H, yields H,
// otherwise substitution will fail.
// This is used as the default return type deducer for std::call.
// Ultimately this may be left as an implementation detail and not exposed,
// but it may be a generally useful facility to specify for users.
template <class H, class... T>
using same_type_or_fail = /*...*/;

// Invoke "fun" with the generated argument list.
// ReturnTypeDeducer is passed along to each provider's
// "provide" function (described later).
// Substitution will fail if the call to "fun" with any of the
// possible generated argument lists would fail substitution.
template <template <class...> class ReturnTypeDeducer = same_type_or_fail,
          class Fun, class... Providers>
constexpr auto call(Fun&& fun, Providers&&... providers) noexcept(/*deduced*/ -> /*deduced*/;

// Invoke "fun" with the generated argument list.
// This is equivalent to invoking std::call with a ReturnTypeDeducer that
// always yields "ReturnType".
template <class ReturnType, class Fun, class... Providers>
constexpr auto call(Fun&& fun, Providers&&... providers) noexcept(/*deduced*/ -> /*deduced*/;

// All "argument providers" are an instantiation of this template.
// The Provider argument must be a type that has a static member function
// template called "provide" that is compatible with the following:
//
//     template <template <class...> class ReturnTypeDeducer, class Fun, class Self>
//     static constexpr auto provide(Fun&& fun, Self&& self) -> /*implementation-dependent*/;
//
// The provide function is the customization point that describes how
// an argument provider generates its portion of the argument list.
// The developer of the provide function does this by invoking
// the function object "fun" with any number of arguments of any type.
// The result of that function call should be returned by provide.
// In the case where the provide function may invoke "fun" with different
// arguments depending on some runtime condition (such as when implementing
// an argument provider that accesses the active field of a variant), then
// ReturnTypeDeducer must be instantiated with the decltype of the result of
// each possible call, and the type that is yielded must be used as the
// return type of the provide function.
//
// "Self" here is a cv-reference-qualified Provider. The provide function
// is static so that it is easy for users to properly forward internal data
// without the need for the user to write multiple overloads.
template <class Provider>
struct argument_provider { Provider /*unspecified*/; };

// An argument provider that evaluates a user-specified provider and forwards
// those arguments along to the user-specified callable. This is used

```

```
// for argument provider composition (such as fully unpacking a tuple of
// tuples). Instances of this are the result of the | operator used in the
// earlier examples and shown below.
template <class Provider, class Callable>
using composed_argument_provider = argument_provider</*unspecified*/>;

// Creates a composed argument provider.
// There should be an overload where the left operand is a reference-to-const
// and also an overloaded where the left operand is an rvalue reference.
template <class Provider, class Callable>
constexpr composed_argument_provider<argument_provider<Provider>&, Callable>
operator |(argument_provider<Provider>&& provider,
          Callable&& next_function) noexcept;
```

The above specification details the core parts of the facility. Below is a small set of suggested argument providers to be included with the facility.

- `unpack(T&& tuple)`: Perfect-forwards the N elements of a tuple as N arguments.
- `active_field_of(V&& var)`: Perfect-forwards the active field of a variant.
- `access_tuple(T&& var, std::size_t index)`: Perfect-forwards the element of the tuple at position "index".
- `to_constant_in_range<class T, T Begin, T End>(U&& value)`: Takes a runtime value and provides the corresponding `std::integral_constant`.
- `to_constant_discriminator<class Variant>(std::size_t discriminator)`: Equivalent to `to_constant_in_range` where the range is `[0, variant-arity)`.
- `to_constant_tuple_index<class Tuple>(std::size_t index)`: Equivalent to `to_constant_in_range` where the range is `[0, tuple-size)`.

These argument providers have all been used at the top level of the examples presented in this paper, with the exception of `std::to_constant_in_range`. This argument provider is proposed because it is useful internally for most argument providers that produce different arguments depending on a runtime value (such as `std::active_field_of`, `std::access_tuple`, and `std::to_constant_discriminator`). Because of this, it should be considered important as a means for people to more easily construct their own argument provider types.

The `std::call` facility opens the door for limitless kinds of argument providers, though only a small handful were presented. It is expected that if this facility is accepted, user-space argument providers would be developed and used. The following is a selection of additional general-purpose argument providers that are useful, but not essential to the most common motivating cases and so they are not currently proposed. This is not an exhaustive list:

- `group(T&&... args)`: Perfect-forwards `args` (useful with the `|` operator).
- `identity(T&& arg)`: Perfect-forwards `arg` (same as `group` with one argument, analogous uses to `boost::protect`).
- `fan(P&& provider, C&&... callables)`: Generates the provider's arguments and passes all of those arguments to each of `callables`.
- `eat(T&&... args)`: Produces an empty list of arguments (useful during composition).

Implementation Samples

Because the implementation of these facilities may not be immediately obvious, the following are example definitions of a few of the facilities.

Example `std::call` Implementation

The following is a simplified definition of `std::call` lacking `noexcept` deduction, desirable SFINAE behavior, and special-casing for void returns.

```
template <class Provider>
struct argument_provider { Provider impl; };

// Implementation details
namespace __detail {

// A trait used internally to either expand an argument provider into its
// generated arguments, or directly forward an argument along if it is not
// an instantiation of argument_provider.
// The default-definition here is the fall-back for when a given argument
// is not an instantiation of argument_provider.
template <class T>
struct argument_provider_traits {
    template <template <class...> class ReturnTypeDeducer, class Fun, class U>
    static constexpr decltype(auto) provide(Fun&& fun, U&& arg) {
        return std::forward<Fun>(fun)(std::forward<U>(arg));
    }
};
```

```

    }
};

// The partial specialization of the above trait for an argument_provider,
// which just forwards the invocation to the user-provided customization point.
template <class Provider>
struct argument_provider_traits<argument_provider<Provider>> {
    template <template <class...> class ReturnTypeDeducer, class Fun, class U>
    static constexpr decltype(auto) provide(Fun&& fun, U&& arg) {
        return Provider::template provide<ReturnTypeDeducer>(std::forward<Fun>(fun),
                                                             std::forward<U>(arg).impl);
    }
};

// Encapsulates a template that can be used as a ReturnTypeDeducer that
// always yields T.
template <class T>
struct always_return {
    template <class...>
    using type = T;
};

} // End __details namespace

// The terminating case of the call function with an explicitly-specified
// return type and when "fun" is invoked with no arguments.
template <class ReturnType, class Fun>
constexpr ReturnType call(Fun&& fun) {
    return std::forward<Fun>(fun)();
}

// The terminating case of the call function with an explicitly-specified
// ReturnTypeDeducer and when "fun" is invoked with no arguments.
template <template <class...> class ReturnTypeDeducer = same_type_or_fail,
        class Fun>
constexpr decltype(auto) call(Fun&& fun) {
    // Just call the function, being sure to use the return type deducer.
    return std::call<ReturnTypeDeducer<decltype(std::declval<Fun>())>>>(
        std::forward<Fun>(fun));
}

// Primary, recursive definition when "call" is given a ReturnTypeDeducer and
// some number of arguments or argument_providers >= 1.
template <template <class...> class ReturnTypeDeducer = same_type_or_fail,
        class Fun, class Head, class... Tail>
constexpr decltype(auto) call(Fun&& fun, Head&& head, Tail&&... tail) {
    return __detail::argument_provider_traits<std::decay_t<Head>>
        ::template provide<ReturnTypeDeducer>(
            // The customization point for "head" may use "head" to provide any
            // number of arguments. It communicates the generated arguments by
            // passing those arguments to the lambda that we give it here.
            [&fun, &tail...](auto&&... expanded_head) -> decltype(auto) {
                // The lambda we give it recurses into "call" with a lambda that
                // captures those arguments that were generated by "head". It takes
                // as parameters the result of the expanded tail arguments.
                return std::call<ReturnTypeDeducer>(
                    [&fun, &expanded_head...](auto&&... expanded_tail) -> decltype(auto) {
                        // At this point, we have the fully generated argument list,
                        // so we can invoke the original function.
                        return std::invoke(
                            std::forward<Fun>(fun),
                            std::forward<decltype(expanded_head)>(expanded_head)...,
                            std::forward<decltype(expanded_tail)>(expanded_tail)...);
                    },
                    std::forward<Tail>(tail)...);
            },
            std::forward<Head>(head));
}

// Primary, definition when "call" is given an explicit return type and
// some number of arguments or argument_providers >= 1.
// This just invokes std::call with a ReturnTypeDeducer that always
// yields ReturnType.
template <class ReturnType, class Fun, class Head, class... Tail>
constexpr decltype(auto) call(Fun&& fun, Head&& head, Tail&&... tail) {
    return std::call<__detail::always_return<ReturnType>::template type>(
        std::forward<Fun>(fun), std::forward<Head>(head), std::forward<Tail>(tail)...);
}

```

std::unpack Implementation

The following is an example implementation of `std::unpack` using `std::apply` internally for brevity and eliding SFINAE exploitation and conditional `noexcept`. The code below is the type that would be used as a template parameter to `std::argument_provider`.

```
template <class T>
struct unpack_impl {
    template <template <class...> class ReturnTypeDeducer, class Fun, class U>
    static constexpr decltype(auto) provide(Fun&& fun, U&& arg) {
        return std::apply(std::forward<Fun>(fun), std::forward<U>(arg).tup);
    }

    T&& tup;
};
```

std::access_tuple Implementation

The following is an example implementation of accessing a tuple with a runtime value. Internally it uses `std::to_constant_tuple_index`, which is built on `std::to_constant_in_range`. `std::to_constant_in_range` does the heavy lifting for this and other argument providers that depend on runtime data. Its implementation will be shown later.

```
template <class T, class I>
struct access_tuple_impl {
    template <template <class...> class ReturnTypeDeducer, class Fun, class Self>
    static constexpr decltype(auto) provide(Fun&& fun, Self&& self) {
        return std::call<ReturnTypeDeducer>(
            [&fun, &self](auto const index_constant) -> decltype(auto) {
                return std::forward<Fun>(fun)(std::get<index_constant.value>(std::forward<Self>(self).tup));
            },
            std::to_constant_tuple_index<std::remove_reference_t<T>>>(
                std::forward<Self>(self).index));
    }

    T&& tup;
    I&& index;
};
```

std::to_constant_in_range Implementation

The following is an example implementation of `std::to_constant_in_range`, which is used behind-the-scenes by `std::access_tuple`, `std::active_field_of`, `std::to_constant_discriminator`, and `std::to_constant_tuple_index`. Once again, this code elides conditional `noexcept` and SFINAE exploitation for brevity.

```
// A function that invokes the provided function with
// a std::integral_constant of the specified value and offset.
template <class ReturnType, class T, T Value, T Offset, class Fun>
constexpr ReturnType invoke_with_constant_impl(Fun&& fun) {
    return std::forward<Fun>(fun)(
        std::integral_constant<T, Value + Offset>());
}

// Indexes into a constexpr table of function pointers
template <template <class...> class ReturnTypeDeducer,
        class T, T Offset, class Fun, class I, I... Indices>
constexpr decltype(auto) invoke_with_constant(Fun&& fun, T index,
        std::integer_sequence<I, Indices...>) {
    // Each invocation may potentially have a different return type, so we
    // need to use the ReturnTypeDeducer to figure out what we should
    // actually return.
    using return_type
        = ReturnTypeDeducer<
            decltype(std::declval<Fun>()(std::integral_constant<T, Indices + Offset>()))...>;

    return std::array<return_type(*)>(Fun&&), sizeof...(Indices){
        {{invoke_with_constant_impl<return_type, T, Indices, Offset, Fun>...}}
        [index - Offset](std::forward<Fun>(fun));
    }
};
```

```
template <class T, T BeginValue, T EndValue>
struct to_constant_in_range_impl {
    // Instantiations of "type" are used as the Provider
    // template argument of argument_provider.
    template <class U>
    struct type
```



```

{
    template <template <class...> class ReturnTypeDeducer, class Fun, class Self>
    static constexpr decltype(auto) provide(Fun&& fun, Self&& self) {
        return __detail::invoke_with_constant<ReturnTypeDeducer, T, BeginValue>(
            std::forward<Fun>(fun),
            std::forward<Self>(self).value,
            std::make_index_sequence<EndValue - BeginValue>());
    }

    U&& value;
};
};

```

Further Work

The specification presented in this proposal is not yet sufficient and will require more effort if the facilities are considered useful. One notable area that needs work is a specification of the requirements of a `ReturnTypeDeducer`. It is also likely that, if accepted, argument providers should live in their own namespace so as to not conflict with functionality in the top-level `std` namespace.

Alternative Customization

The current specification requires that all argument providers are instantiations of the `argument_provider` template. This was done for simplicity, but customization could be done equivalently with traits directly. The author of this proposal leaves the means of customization open for discussion if it is a point of contention.

Acknowledgments

Thanks to Tony Van Eerd who encouraged me to write this paper, and to Michael Park who pointed out the `constexpr` table-lookup form of variant visitation to me, which is used in the example implementation of `std::to_constant_in_range`.

References

- [1] Axel Naumann: "Variant: a type-safe union that is rarely invalid" P0088R0 <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/p0088r0.pdf>
- [2] Python Software Foundation: "The Python Tutorial" <https://docs.python.org/3.5/tutorial/controlflow.html#unpacking-argument-lists>
- [3] Clay Labs: "The Clay Programming Language, Language Reference" <https://github.com/jckarter/clay/blob/master/doc/language-reference.md#dispatchoperator>
- [4] Steven Watanabe: "Boost.Switch" <http://lists.boost.org/boost-announce/2008/01/0166.php>